

## 1. Biến Fourier rời rạc (DFT / FFT)

### Công thức DFT 2D:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

### IDFT 2D:

$$f(x, y) = \frac{1}{MN} \sum_u \sum_v F(u, v) e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

```
FFT bằng NumPy
F = np.fft.fft2(img)          # DFT 2D -> số phức
Fs = np.fft.fftshift(F)        # DC ra tâm
S = np.log(1 + np.abs(Fs))     # Phổ biên độ (log scale)
```

#### Lưu ý:

- fftshift: dịch DC từ góc ra **chờnh gia**
- Đúng  $\log(1+|F|)$ : DC ở trung tâm, log cho thấy tần suất
- DC = giá trị trung bình tất cả pixel  $\Rightarrow$  ảnh sẫm/nhạt tóe
- nhều texture  $\Rightarrow$  phổ lan rộng ra xa tâm hơn

phép tính: DFT có độ phức tạp  $O(M^2N^2)$ ; FFT  $O(MN \log MN)$

## 2. Lọc trong miền tần số

```
# Lọc thông thấp lý tưởng (Ideal LPF)
def circular_lpf_mask(shape, r):
    M, N = shape
    crow, ccol = M//2, N//2
    Y, X = np.ogrid[:M, :N]
    dist2 = (Y-crow)**2 + (X-ccol)**2
    return (dist2 <= r*r).astype(np.float64)
```

```
Hlp = circular_lpf_mask(img.shape, r=30)
Hhp = 1 - Hlp  # HPF = 1 - LPF
```

```
# Áp dụng bộ lọc
def apply_freq_filter(image, mask):
    F = np.fft.fft2(image)
    Fs = np.fft.fftshift(F)
    Gs = Fs * mask  # Nhân phổ với mặt nạ
    g = np.fft.ifft2(np.fft.ifftshift(Gs))
    return np.abs(g)
```

| $r$     | LPF     | HPF          |
|---------|---------|--------------|
| Nh (10) | Rất mịn | Chỉ nhiễu mờ |
| Va (30) | Mịn vừa | Bảo vệ rõ    |
| Ln (60) | Gần gốc | Chỉ nhiễu    |

**Hiện tượng Ringing (Gibbs):** Ideal LPF cắt đột ngột  $\Rightarrow$  xung quanh biên. Gaussian LPF mượt hơn, ít ringing hơn.

## 3. Tách kênh rời rạc 2D

Công thức:  $(f * h)(x, y) = \sum_i \sum_j f(x-i, y-j) \cdot h(i, j)$

nhân lý tách kênh:  $f * h \iff F \cdot H$

```
# Tu chỉnh dữ liệu (same padding, zero-pad)
def conv2d_same(image, kernel):
    kh, kw = kernel.shape
    ph, pw = kh//2, kw//2
```

```
padded = np.pad(image, ((ph, ph), (pw, pw)), 'constant')
H, W = image.shape
out = np.zeros((H, W), dtype=np.float64)
for i in range(H):
    for j in range(W):
        out[i, j] = np.sum(padded[i:i+kh, j:j+kw]*kernel)
return out

# OpenCV (BORDER_REFLECT_101 mặc định)
out_cv = cv2.filter2D(image, -1, kernel)
```

**Sai khác với biên:** zero-padding  $\rightarrow$  viền tối;  
cv2.filter2D đúng reflect  $\rightarrow$  viền nhàn hơn.

Viền trong ảnh bị ảnh hưởng bởi  
k\_mean = np.ones((5,5)) / 25.0 # NumPy  
out\_cv = cv2.blur(image, (5,5)) # OpenCV  
**4. Các bộ lọc trung bình**  
Trung bình (Mean / Box Filter)

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

```
out = cv2.GaussianBlur(image, (ksize, ksize), sigma)
# sigma=0 -> tự động; sigma=0.5 -> mịn; sigma=2.0 -> mờ rõ
```

|          | Mean  | Gaussian     |
|----------|-------|--------------|
| Kernel   | u     | Gim đơn giản |
| Giả định | Không | Tốt hơn      |

```
out = cv2.medianBlur(image, ksize) # ksize phải lẻ: 3,5,7,...
```

**Khi nào dùng Median?** Nhiều Salt & Pepper (nhiều xung).

**Median robust với outlier:** giá trị 0/255 lấn át mức 2 giá trị xung quanh  $\Rightarrow$  loại bỏ hoàn toàn. Mean/Gaussian chỉ lan nhiễu xung quanh.

```
# Thêm nhiễu Gaussian
def add_gaussian_noise(image, sigma=20):
    noise = np.random.normal(0, sigma, image.shape)
    return np.clip(image.astype(np.float64)+noise, 0, 255).astype(np.uint8)

# Thêm nhiễu Salt & Pepper
def add_salt_pepper(image, prob=0.02):
    out = image.copy()
    rnd = np.random.rand(*image.shape)
    out[rnd < prob/2] = 0
    out[rnd > 1-prob/2] = 255
    return out
```

| Nhiều          | Bộ lọc nhiễu | Lý do              |
|----------------|--------------|--------------------|
| Gaussian       | GaussianBlur | Cứng phớt lờ       |
| Salt & Pepper  | MedianBlur   | Robust với outlier |
| Gaussian nhiễu | Mean filter  | Nhiều, hiệu quả    |

### Phép toán 1: Laplacian

$$\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad g = f - \lambda \nabla^2 f$$

```
k_lap = np.array([[0,1,0],[1,-4,1],[0,1,0]], dtype=np.float64)
lap = conv2d_same(image, k_lap)
sharp = np.clip(image.astype(np.float64) - lam*lap, 0, 255).astype(np.uint8)
# lam=0.3: nhẹ; lam=0.7: vừa; lam>=1.2 -> artifact rõ
```

### Phép toán 2: Unsharp Masking (khuyến nghị)

$$g = f + k \underbrace{(f - f_{\text{blur}})}_{\text{high-freq mask}}$$

```
blur = cv2.GaussianBlur(image, (5,5), 1.0)
hf = image.astype(np.float64) - blur.astype(np.float64)
sharp = np.clip(image.astype(np.float64) + k*hf, 0, 255).astype(np.uint8)
# k=0.5: nhẹ; k=1.0: vừa; k>=1.5 -> overshoot
```

|             | Laplacian           | Unsharp Masking           |
|-------------|---------------------|---------------------------|
| Nhiều nhiễu | Rất nhiễu (bc 2)    | Ồn nhiễu (blur lọc nhiễu) |
| Artefact    | Halo mờ             | Overshoot nhiễu           |
| Đúng        | Cần chỉnh $\lambda$ | Ồn nhiễu rõ hơn           |

## 8. Thuật toán Sobel

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

$$\|\nabla f\| = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan 2(G_y, G_x)$$

```
Gx = np.array([[ -1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float64)
Gy = np.array([[ -1, -2, -1], [ 0, 0, 0], [ 1, 2, 1]], dtype=np.float64)
Sx = conv2d_same(img, Gx); Sy = conv2d_same(img, Gy)
mag = np.sqrt(Sx**2 + Sy**2)
# Ngưỡng hóa
edge_map = (mag >= T).astype(np.uint8) * 255
# T=30: nhiều biên; T=60: cân bằng; T=90: sạch but dứt
```

### 0. Thuật toán Canny (5 bước)

#### 5 bước Canny:

- Gaussian blur — loại nhiễu
- Sobel gradient — tính + hướng biên
- NMS — lọc nhiễu biên (chỉ giữ 1)
- Double threshold — phân loại strong/weak
- Hysteresis — kết nối weak  $\rightarrow$  strong

```
edges = cv2.Canny(image, threshold1=30, threshold2=80)
```

```
NMS (Non-Maximum Suppression)
def non_maximum_suppression(G, theta):
    angle = theta * 180/np.pi
    angle[angle < 0] += 180
    Z = np.zeros_like(G)
    for i in range(1, G.shape[0]-1):
        for j in range(1, G.shape[1]-1):
```

```
if (0<=angle[i,j]<22.5) or (157.5<=angle[i,j]<=180):
    q,r = G[i,j+1], G[i,j-1] # 0 do
elif 22.5<=angle[i,j]<67.5:
    q,r = G[i+1,j-1], G[i-1,j+1] # 45 do
elif 67.5<=angle[i,j]<112.5:
    q,r = G[i+1,j], G[i-1,j] # 90 do
else:
    q,r = G[i-1,j-1], G[i+1,j+1] # 135 do
if G[i,j]>=q and G[i,j]>=r:
    Z[i,j] = G[i,j]

return Z
```

```
def double_threshold(img, t_low, t_high, weak=50, strong=255):
    res = np.zeros_like(img, dtype=np.uint8)
    res[img >= t_high] = strong
    res[(img >= t_low) & (img < t_high)] = weak
    return res
```

```
def hysteresis(img, weak=50, strong=255):
    out = img.copy()
    for i in range(1, out.shape[0]-1):
        for j in range(1, out.shape[1]-1):
            if out[i,j] == weak:
                out[i,j] = strong if np.any(
                    out[i-1:i+2,j-1:j+2]==strong) else 0
    return out
```

| $(T_l, T_h)$ | Biển           | Nhiu       |
|--------------|----------------|------------|
| (20,60)      | Nhiu, lỏng tç  | Nhiu nhiu  |
| (30,80)      | <b>Cón bng</b> | Va phi     |
| (50,120)     | Ồt, sch        | Ồt, nhng t |

**Vớ sao hysteresis tt hn ngng n?**

Gi weak edge **ch** khi nú kt nì vì strong edge ⇒ loi nhiu cũ lp, biển lỏng tç hn.

**10. So sánh Spatial vs Frequency Filtering**

|               | Spatial         | Frequency          |
|---------------|-----------------|--------------------|
| Thao tọc      | Kernel trt      | Nhón ph            |
| Gaussian blur | GaussianBlur    | Gaussian LPF       |
| Gìng nhau?    | Khũng hoịn toịn | (ideal ≠ Gaussian) |
| Trc quan      | D               | Cn bit FFT         |
| ng dng        | Thũng dng       | Lc chn lc          |

Gaussian spatial blur ≠ Circular LPF vớ:  
Circular LPF lý tng ct ngng t ngт ⇒ ringing. Gaussian blur  
↔ Gaussian LPF (mt, khũng ringing).

**11. Mini-Project: Pipeline hoịn chnh**

nh gc → Nhiu → Kh nhiu → Sc nốt → Biển

```
img_clean = img_edge.copy() # anh sach ban dau

# === Pipeline 1: Gaussian noise ===
noisy_g = add_gaussian_noise(img_clean, sigma=25)
denoised_g = cv2.GaussianBlur(noisy_g, (5,5), 1.0)
blur_tmp = cv2.GaussianBlur(denoised_g, (5,5), 1.0)
hf = denoised_g.astype(np.float64) - blur_tmp.astype(np.float64)
sharp_g = np.clip(denoised_g.astype(np.float64)+1.0*hf, 0,255).
    astype(np.uint8)
edges_g = cv2.Canny(sharp_g, 30, 80)

# === Pipeline 2: Salt & Pepper noise ===
noisy_sp = add_salt_pepper(img_clean, prob=0.03)
denoised_sp = cv2.medianBlur(noisy_sp, 5)
blur_tmp2 = cv2.GaussianBlur(denoised_sp, (5,5), 1.0)
hf2 = denoised_sp.astype(np.float64) - blur_tmp2.astype(np.float64)
sharp_sp = np.clip(denoised_sp.astype(np.float64)+1.0*hf2, 0,255).
    astype(np.uint8)
edges_sp = cv2.Canny(sharp_sp, 30, 80)
```

| Nhiu                 | Kh           | Sc nốt        | Biển         |
|----------------------|--------------|---------------|--------------|
| Gaussian $\sigma=25$ | GaussianBlur | Unsharp $k=1$ | Canny(30,80) |
| S&P $p=0.03$         | MedianBlur   | Unsharp $k=1$ | Canny(30,80) |

**Trade-off 3 chiu — Quy tç vịnh:**

- Kh nhiu mnh ⇒ ồt nhiu nhng **mt biển yu**
- Sc nốt mnh ⇒ rủ biển nhng **khuch i nhiu**
- Ngng Canny cao ⇒ sch nhng **biển t on**

⇒ **Khũng cú tham s “tt nht” chung**, phi cón bng c 3 cho tng nh!

**12. Bng hịm OpenCV nhanh**

| Hịm                           | Mc ớch            |
|-------------------------------|-------------------|
| cv2.blur(img,(k,k))           | Lc trung bớnh     |
| cv2.GaussianBlur(img,(k,k),s) | Lc Gaussian       |
| cv2.medianBlur(img,k)         | Lc trung v        |
| cv2.filter2D(img,-1,kernel)   | Tồch chp tng quồt |
| cv2.Canny(img,t1,t2)          | Dù biển Canny     |
| np.fft.fft2(img)              | FFT 2D (complex)  |
| np.fft.fftshift(F)            | DC ra tótm        |
| np.fft.ifft2(F)               | IFFT 2D           |
| np.fft.ifftshift(F)           | o fftshift        |

**13. Cũng thc cn nh & Tips**

| Tổn           | Cũng thc                                              |
|---------------|-------------------------------------------------------|
| DFT           | $F(u,v) = \sum_x \sum_y f e^{-j2\pi(ux/M+vy/N)}$      |
| Tồch chp      | $(f * h)(x,y) = \sum_i \sum_j f(x-i,y-j)h(i,j)$       |
| Conv. theorem | $f * h \Leftrightarrow F \cdot H$                     |
| Gaussian      | $G = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$ |
| Gradient      | $\ \nabla f\  = \sqrt{G_x^2 + G_y^2}$                 |
| Unsharp       | $g = f + k(f - f_{\text{blur}})$                      |
| Laplacian     | $g = f - \lambda \nabla^2 f$                          |

**7 tips thc hịnh:**

- Luũn chuyñ uint8 → float64 trc khi tồnh
- np.clip(result,0,255).astype(np.uint8) trồnh trịn s
- Ph FFT đứng log(1+|F|), khũng DC nut ht
- Zero-pad → vin ti; reflect-pad → t nhĩỏn hn
- So sỏnh NumPy vs OpenCV: np.abs(a-b) diff map
- Kh nhiu TRC** khi dù biển — khũng bao gì ngc
- Median cho S&P, Gaussian cho Gaussian noise** — quy tç vịnh